Predicting Wellness Package Buyers

MLOps pipeline on GitHub Actions · Advanced Machine Learning and MLOps

Deakin Applied AI Master Bridge Program · Mohammad Alrashed · 23 October 2026 · v1.2



Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Contents / Agenda

- Executive Summary
- Data Registration
- Data Preparation
- Model Building with Experimentation Tracking
- Model Deployment
- Automated GitHub Actions Workflow
- Output Evaluation
- Appendix

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Executive Summary

- **Problem**: “Visit with Us” picks customers to call by hand. Only 19% of contacted customers bought a package.
- **Solution**: an XGBoost model scores each customer’s chance to buy. GitHub Actions retrains it on every push and commits the new model. Two live apps serve it (Streamlit Community Cloud and Hugging Face), kept awake by a scheduled job.
- **Result**: on unseen customers the model finds 82% of buyers, and 86% of its “will buy” calls are right (F1 0.84, ROC-AUC 0.96).
- **Recommendations**:
  - Call the top-scored 20% of leads first. That list holds 86% of all buyers.
  - Target passport holders, single and younger (≤25) customers, and Executives. They buy 2–3× more than average.
  - Use the Basic package as the entry offer. Higher tiers convert at 8–16%.
  - Plan 5–6 follow-ups for warm leads. Buy rate rises from 17% at 3 calls to 40% at 6.

0.84

Test F1 (buyers)

0.96

Test ROC-AUC

86%

of buyers in top-20% scored leads

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Data Registration

- **Master folder**: tourism_project/ holds everything the pipeline needs.
  - data/ keeps tourism.csv under version control.
  - model_building/ holds data_register.py, prep.py and train.py.
  - deployment/ holds app.py, its requirements and the model the pipeline commits.
- **Registration**: the dataset lives inside the public GitHub repo. No external data store is needed.
- **data_register.py**: checks the file exists, checks all 20 expected columns and a binary target, then logs a SHA-256 hash, the shape and the class counts.
- **In GitHub Actions**: the register-dataset job runs the script and uploads tourism.csv as the registered-data artifact.

```
Deakin-AIML-Bridge-Tourism-
  Wellness-MLOps/ (public)
├─ .github/workflows/
│   ├─ pipeline.yml
│   └─ keepalive.yml
└─ tourism_project/
   ├─ requirements.txt
   ├─ data/
   │   └─ tourism.csv
   ├─ model_building/
   │   ├─ data_register.py
   │   ├─ prep.py
   │   └─ train.py
   └─ deployment/
       ├─ app.py
       ├─ requirements.txt
       └─ best_tourism_model_v1.joblib
```

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Data Preparation

- **Load**: prep.py reads tourism_project/data/tourism.csv from the checked-out repo.
- **Remove columns**: “Unnamed: 0” (a saved index) and CustomerID (a unique key) carry no signal.
- **Clean**: merge the Gender typo “Fe Male” into Female. Drop 117 exact duplicate rows so no record sits in both train and test. Impute missing values (none in this release).
- **Split**: 80/20, stratified on ProdTaken, random_state = 42. Both sets keep the 19.3% buyer rate.
- **Hand-off**: Xtrain, Xtest, ytrain and ytest CSVs are uploaded as the data-splits artifact for the training job. This replaces the Hugging Face upload in the original brief.

Stage	Rows	Columns
Raw dataset	4,128	21
After column drop	4,128	19
After de-duplication	4,011	19
Train (80%)	3,208	18 + target
Test (20%)	803	18 + target

Buyer rate: 19.3% in both train and test.
Features: 12 numeric, 6 categorical.

Link to Appendix slide on data background check

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Model Building with Experimentation Tracking

- **Inputs**: the data-splits artifact from the data-prep job.
- **Pipeline**: StandardScaler on numeric and OneHotEncoder on categorical columns, then XGBClassifier with scale_pos_weight ≈ 4.18 for the 19% buyer class.
- **Tuning**: GridSearchCV over 24 combinations (n_estimators, max_depth, learning_rate, colsample_bytree), stratified 5-fold CV, F1 scoring.
- **Tracking**: every combination is logged to MLflow as a nested run (params, mean and std CV F1). The parent run logs best params and test metrics.
- **Best**: max_depth 7, 200 trees, learning_rate 0.1, colsample 0.7. CV F1 0.781.
- **Commit**: best_tourism_model_v1.joblib, model_metadata.json and tuning_results.csv are pushed back to deployment/.

Metric	Train	Test
Accuracy	1.000	0.940
Precision	1.000	0.864
Recall	1.000	0.819
F1	1.000	0.841
ROC-AUC	1.000	0.961

Test confusion matrix (803 customers)

	Pred. no	Pred. buy
Actual no	628	20
Actual buy	28	127

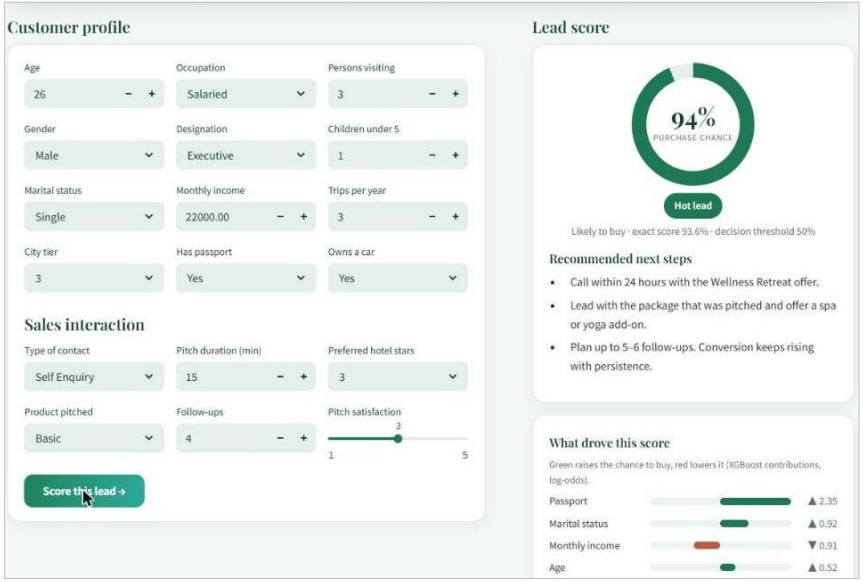
Train scores of 1.0 show overfitting. Test scores stay strong, so v1 ships, with regularisation queued for v2.

Link to Appendix slide on data background check

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Model Deployment

- **app.py**: loads the model and model_metadata.json committed by the pipeline. Paths are relative to the file, so it runs the same on both hosts.
- **Inputs**: 18 fields go into a one-row DataFrame with the training column names. The saved pipeline scales and encodes them itself.
- **Design**: tourism hero banner, form on the left, lead score on the right: probability ring, Hot / Warm / Cool tier, next steps, and the top factors from XGBoost contributions.
- **Dependencies**: deployment/requirements.txt pins streamlit, scikit-learn, xgboost, pandas, numpy and pyarrow, so the model loads with its training versions.
- **Hosting**: Streamlit Community Cloud (main, app.py, Python 3.11) and a public Hugging Face Docker Space that clones the repo and pulls main on every start.
- **Always on**: keepalive.yml opens both apps every 6 hours for the 60-day review window.

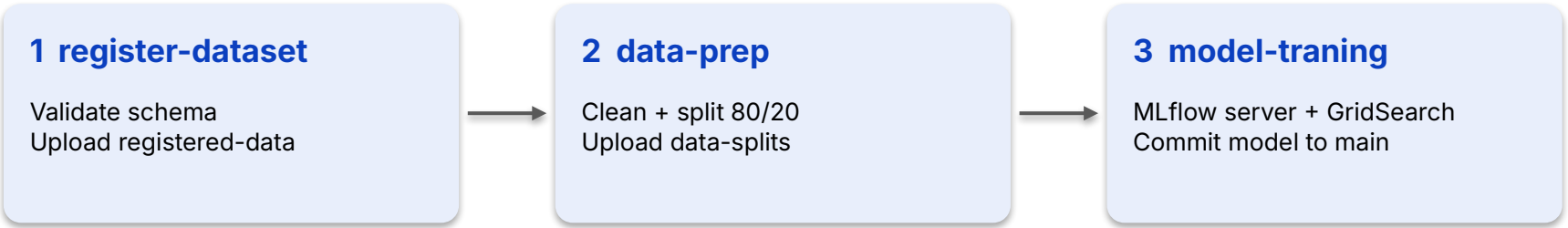


Live Streamlit Cloud app: same customer scores 93.6%, a Hot lead.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Automated GitHub Actions Workflow

- **pipeline.yml**: lives in .github/workflows/. It runs on every push to main and on manual “Run workflow”.
- **Three chained jobs**: each uses Python 3.11 and installs tourism_project/requirements.txt. Data moves between jobs as artifacts.
- **Auto-update**: the last step commits the new model to main with “[skip ci]”, so the commit does not retrigger the pipeline. permissions: contents: write lets the bot push. A second workflow, keepalive.yml, keeps both apps awake.

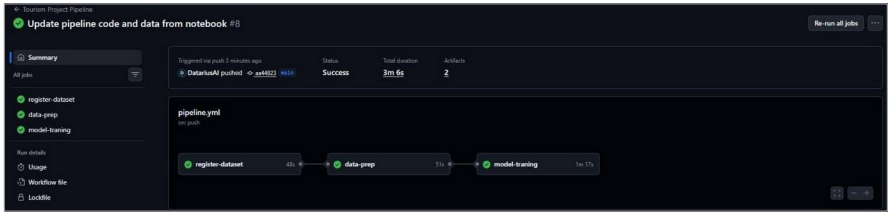


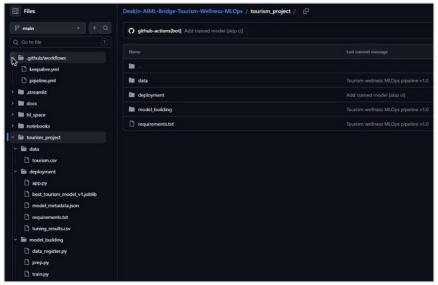
Job name “model-traning” is kept as given in the course template.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Output Evaluation

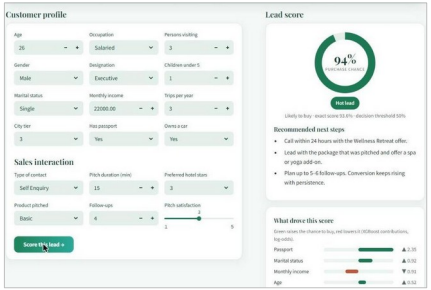
GitHub (public): https://github.com/DatariusAI/Deakin-AIML-Bridge-Tourism-Wellness-MLOps
Streamlit Community Cloud: https://datariusai-tourism-wellness-mlops.streamlit.app
Hugging Face Space: https://huggingface.co/spaces/DatariusAI/tourism-wellness-mlops



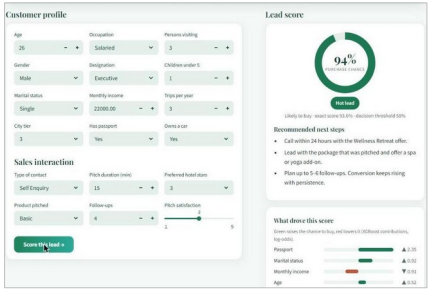




Run #8, started by the notebook push: 3 jobs green in 3m 6s. The bot commits the model.
Keep-alive run: both apps awake.



Streamlit Community Cloud



Hugging Face Space

Same customer on both apps: 26, single, Executive, passport, tier-3 city, Basic package.
Both return 93.6%, a Hot lead, so both front ends serve the same pipeline-trained model.

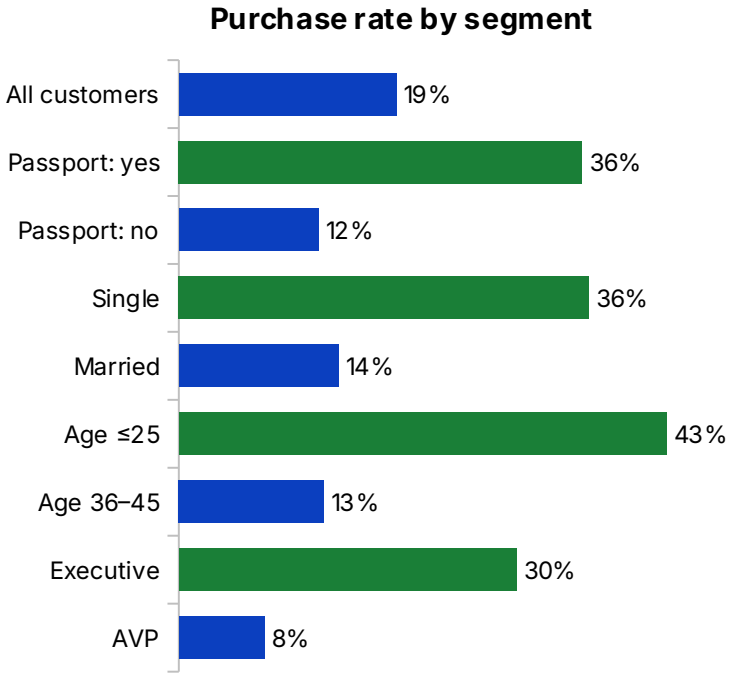
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

APPENDIX

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Dataset Overview and Registration

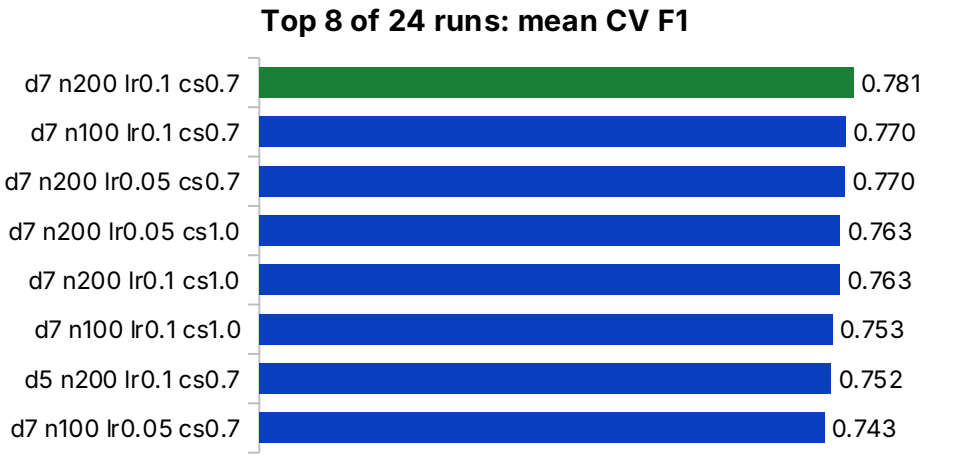
- **Source**: “Visit with Us” customer and sales-interaction records (tourism.csv, course-provided).
- **Size**: 4,128 customers × 20 fields, no missing values. 117 exact duplicates were removed.
- **Target**: ProdTaken (1 = bought). 797 buyers (19.3%), so the classes are imbalanced.
- **Features**: customer profile (Age, CityTier, Occupation, Gender, MaritalStatus, Passport, OwnCar, Designation, MonthlyIncome, trips, party size) and pitch data (TypeofContact, ProductPitched, DurationOfPitch, NumberOfFollowups, PitchSatisfactionScore).
- **Registration**: committed at tourism_project/data/tourism.csv and validated by data_register.py on each run.



Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Experimentation Tracking Details

- **Model**: XGBoost in one sklearn pipeline. 24 combinations × 5 folds = 120 fits.
- **Grid**: n_estimators {100, 200}, max_depth {3, 5, 7}, learning_rate {0.05, 0.1}, colsample_bytree {0.7, 1.0}.
- **Logged per run**: params, mean_cv_f1, std_cv_f1 (nested). Parent run: best params, scale_pos_weight, threshold, train/test metrics and the model artifact.
- **Finding**: depth matters most. Every depth-7 run beats every depth-3 run (0.73–0.78 vs 0.60–0.64 CV F1).



Worst runs	mean CV F1	std
d3 n100 lr0.1 cs0.7	0.611	0.017
d3 n100 lr0.05 cs0.7	0.605	0.027
d3 n100 lr0.05 cs1.0	0.596	0.027

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

GitHub Actions Workflow Details

- **Trigger**: on.push.branches = [main] plus workflow_dispatch.
- **Each job**: checkout@v4 → setup-python@v5 (3.11) → pip install -r tourism_project/requirements.txt.
- **register-dataset**: python data_register.py → upload-artifact registered-data.
- **data-prep**: python prep.py → upload-artifact data-splits (4 CSVs).
- **model-traning**: download data-splits → nohup mlflow ui --port 5000 → python train.py → git add / commit "[skip ci]" / push.
- **Code updates**: the notebook push cell commits tourism_project/, .github/ and .streamlit/ to main in one git push. That push starts the pipeline, so code and model stay in step.
- **keepalive.yml**: cron every 6 h opens both apps in headless Chromium and re-enables itself past GitHub's 60-day limit.

Job · step	Result
register-dataset (0m 48s)	✔ success
Install Dependencies	✔
Register Dataset	✔
Upload Registered Dataset	✔
data-prep (0m 51s)	✔ success
Install Dependencies	✔
Run Data Preparation	✔
Upload Train Test Splits	✔
model-traning (1m 17s)	✔ success
Install Dependencies	✔
Download Train Test Splits	✔
Start MLflow Server	✔
Model Building	✔
Commit Trained Model	✔

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

greatlearning
Power Ahead



Happy Learning !